ShuffleSnap: strided windowed exact reassignment for point-to-grid layout

Kyle McDonald

October 6, 2026

Abstract

Snapping each point of a 2D embedding, such as a UMAP layout, to its own grid cell while minimizing the total squared displacement is a linear assignment problem too large for dense solvers at 10^5 points. ShuffleSnap, a local search inspired by Shellsort, splits the grid into interleaved subgrids at a decreasing sequence of strides and solves small exact assignment problems in shifted windows of each, starting every solve from closed-form optimal-transport duals. Against optima certified by a sparse exact solver that proves optimality for the full, dense problem, the default preset is within 0.06% of the optimum on nine of ten distributions at 10^5 points, in a median of 2.3 s on one core or 0.19 s on 24 threads. The fast and default presets take about as long as BSP-OT with 16 and 64 plans, and at equal time they are closer to the optimum on eight and nine of the ten distributions; RasterFairy is 17–425% above the optimum on seven. The remaining error forms long loops of one-cell shifts that no window can remove, which makes uniform points, concave masks and surplus cells hard.

1 Introduction

A common way to show a large image or document collection is to embed the items in the plane, for example with t-SNE [13] or UMAP [8], and then snap every item to its own cell of a regular grid, so that thumbnails tile the screen without overlap. Snapping is an assignment problem: each point must receive a distinct cell, and points should move as little as possible. Measured as total squared displacement, the best snapping is a linear assignment problem, and also a discrete optimal-transport problem with quadratic cost [10].

For a few thousand points, practitioners solve it exactly with a dense Hungarian or Jonker–Volgenant solver [4, 6]. The dense cost matrix needs $8N^2$ bytes, 80 GB at $N = 100{,}000$, so these solvers stop being usable between roughly 10,000 and 30,000 points. Beyond that, people use recursive splitting, as RasterFairy does [5], or fast approximate transport such as BSP-OT [3].

ShuffleSnap takes its name and its idea from Shellsort [12], which sorts interleaved subsequences at a decreasing sequence of gaps. At stride s, ShuffleSnap splits the grid into s^2 interleaved subgrids and solves small *exact* assignment problems inside 8×8 windows of each, dividing s by $\sqrt{2}$ until it reaches 1 (Figure 1). All work is at full resolution; nothing is aggregated.

We measure every gap against a proven optimum and report where the method fails. We contribute a careful parallel implementation with a 70-line serial reference that reproduces it exactly; a window solver that starts from closed-form transport duals; a certified exact solver that reaches 10^6 points, so that every gap is measured against a proven optimum; measured design choices; a diagnosis of the remaining error, with inputs on which the relative gap is unbounded; and a benchmark against exact solvers, RasterFairy and BSP-OT that keeps displacement and neighbourhood preservation apart.

2 Problem

A target is a boolean mask on a $W \times H$ lattice; its usable positions are the cells $c_1, \dots, c_M$, centered on integer coordinates so that one unit is one cell. Input points are mapped into this frame before anything else, each axis of their bounding box onto that of the cells, and every method receives the same mapped points $p_1, \dots, p_N$ ($N \le M$). We seek an injective assignment σ minimizing

$$C(\sigma) = \sum_{i=1}^{N} \|p_i - c_{\sigma(i)}\|^2. \tag{1}$$

When $N < M$ the solver must also decide which cells stay empty, and the cell term of the cost then depends on that choice. The linear-programming relaxation of Equation (1) has integral optima, because its constraint matrix is totally unimodular, so linear-programming duality gives a certificate of optimality.

Proposition 1. *For any cell potentials* $v \in \mathbb{R}^M$, $\mathrm{LB}(v) = \sum_i \min_j(\|p_i - c_j\|^2 - v_j) + \sum_j v_j - (M - N)\max_j v_j$ *is a lower bound on the optimal cost, and it is tight for optimal potentials.*

Sketch. Shifting v by its maximum gives nonpositive potentials v'; with $u_i = \min_j(c_{ij} - v'_j)$, the pair (u, v') is feasible for the dual of the assignment program, and its dual objective equals $\mathrm{LB}(v)$. Weak duality gives the bound, strong duality its tightness. $\square$

The bound needs a minimum over *all* cells for every point, but it does not depend on how v was found, so $C(\sigma) - \mathrm{LB}(v)$ certifies the gap of any assignment. We report the relative excess $C(\sigma)/C^\star - 1$ over a certified optimum $C^\star$, and where it misleads, the absolute excess per point: points that almost sit on the cells have a tiny optimum, which inflates relative numbers, and heavy-tailed inputs have a huge one, which hides them.

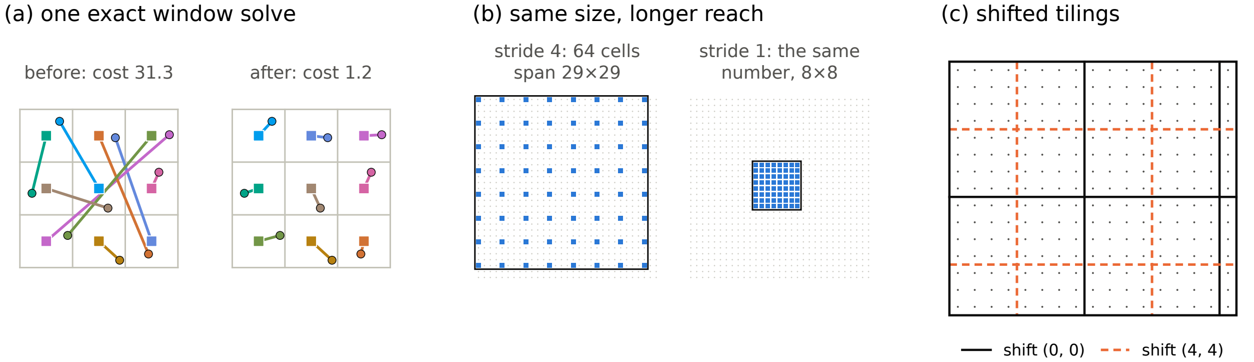
(a) one exact window solve
before: cost 31.3
after: cost 1.2
(b) same size, longer reach
stride 4: 64 cells span 29×29
stride 1: the same number, 8×8
(c) shifted tilings
shift (0, 0)
shift (4, 4)

Figure 1: (a) A window collects the points that occupy its cells and reassigns them among the same cells by an exact linear assignment. (b) At stride s a window takes every s-th row and column: it keeps 64 cells but spans $7s + 1$ cells per axis, so points can travel far. (c) Each subgrid is tiled four times, shifted by half a window, so that points can cross every window border.

3 ShuffleSnap

ShuffleSnap has two main parameters, the window side w and the stride ratio r; the default is $w = 8$ and $r = \sqrt{2}$ (Section 3.1).

1. **Start.** Place the points on a random subset of the cells, as a seeded random permutation.
2. **Strided windows.** At stride (s_x, s_y), lattice positions with the same residues $(x \bmod s_x, y \bmod s_y)$ form one of $s_x s_y$ subgrids. Each subgrid is tiled with windows of $w \times w$ of its cells, spaced s apart and spanning $(w-1)s+1$ cells per axis.
3. **Exact reassignment.** In every window, the points that occupy its cells are reassigned among the same cells, empty ones included, by an exact linear assignment on squared distances to the original coordinates.
4. **Shifted tilings.** The tiling is repeated with shifts $(0, 0)$, $(0, w/2)$, $(w/2, 0)$ and $(w/2, w/2)$ in subgrid coordinates. These four phases form a round; the windows of a phase are disjoint and are solved in parallel.
5. **Schedule.** On an axis of length L the first stride is the smallest power of two with $ws \geq L$. After one round both strides are divided by r and rounded, never below 1. Stride-1 rounds repeat until a round moves no point, or until a time budget runs out.

Acceptance. Each window is a problem with k occupants and $m \geq k$ cells, which we solve exactly by shortest augmenting paths in $O(k^2 m)$ time, from geometric starting duals (Section 3.1). The new assignment replaces the old one only if it is cheaper by more than a relative 10^{-12}. Since the old assignment is feasible for the window, every accepted change lowers the global cost: the cost is monotone, the search terminates, and ties, such as duplicated points, never cause cycling. The final no-move round certifies a fixed point of the windows, not global optimality.

Boundaries, rectangles, masks. Windows that cross a subgrid edge are clipped, never dropped, on all four sides, so every lattice position belongs to exactly one window of every phase and boundary cells trade with the interior; windows with fewer than two usable cells are skipped. Each axis starts from its own stride, and the axis that reaches 1 first stays there. Masked positions never enter a window. Empty usable cells do, so with more cells than points the method also decides which cells stay empty. A prime number of points can therefore fill an exact-fit grid with a partial last row, or a full rectangle with a few surplus cells.

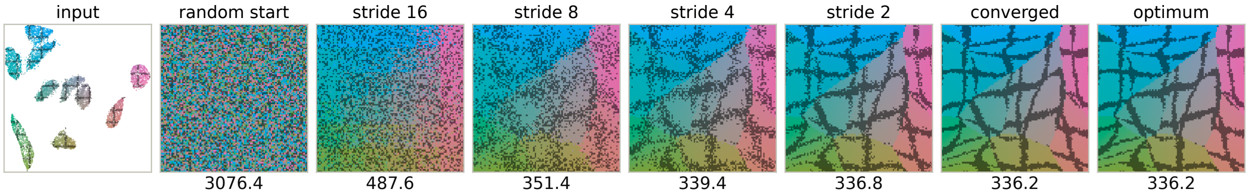

Figure 2: Progress on 10,000 points of a UMAP embedding of MNIST digits [7], default preset, every second stride shown. Every point keeps the color of its input position, and the dark bands are graph-paper lines drawn on the input; both travel with the points, so smooth colors and unbroken lines mean that the input neighbourhoods survived. Numbers are the mean squared displacement in cells^2, next to the certified optimum.

3.1 Implementation and design changes

The kernel is C++17 with nanobind bindings, parallelized with OpenMP over the windows of a phase. Because windows in a phase are disjoint and each solve is deterministic, the result is bit-identical for any thread count. A window whose cells have not changed occupant since it was last solved in the same stage is already optimal and is skipped, which matters in the stride-1 rounds. The exact cost is recorded after every phase, which gives an anytime trace. A serial reference of about 70 lines of NumPy and SciPy reproduces the kernel's assignments exactly and serves as an executable specification. A phase at any stride has about M/w^2 windows of at most w^2 cells, so each level costs about the same, and with $O(\log L)$ levels the schedule costs $O(M \log M)$ plus the convergence rounds; the working memory is 55–58 bytes per point.

We tuned only on development instances and report every claim on held-out ones. *Window size* is the main quality knob: each step from 8×8 to 12×12 to 16×16 cost 2–2.6 times the time and removed one half to two thirds of the remaining excess. The *fast* (5×5) and *balanced* (8×8, default) presets take about as long as BSP-OT with 16 and 64 plans; *quality* is 12×12. *Stride ratio:* dividing by $\sqrt{2}$ instead of 2 was the best or close to the best ratio at every window size. *Start:* structured starts, such as recursive bisection or sorting by rows, are worse than a random one: they create straight seams that windows cannot repair. *Solver:* every point of a distant cluster prefers the same corner cell, which defeats the usual greedy start. Starting from the Brenier potential [2] of the per-axis sorted map instead stays exact and made window solves 2–5 times faster, which buys larger windows at equal time; Jonker–Volgenant and an auction were slower.

4 A certified exact solver

At 10^5 points no dense matrix fits in memory, so we built a solver that never forms the dense problem, yet certifies optimality against it. It gives each point candidate edges: its 12 nearest cells, plus the 5×5 lattice neighbourhood of its cell in a starting assignment, which guarantees that a perfect matching exists. It solves the min-cost flow on these edges with OR-Tools' cost-scaling solver [9], on costs scaled to integers. From that solution it computes cell potentials by shortest paths, in exact integer arithmetic, and evaluates LB (Proposition 1) over *all* cells, scanning blocks of cells only where they can beat the current minimum. The pass also lists every cell whose reduced cost undercuts a point's current cell; if any exists, those edges are added and the flow is solved again. This is the core-and-price idea of sparse assignment and transport solvers [11, 14]. The certificate $C(\sigma) - \mathrm{LB}(v)$ is computed from real-valued costs and holds whatever errors occur along the way; integer rounding can only loosen it, by at most $4N/S$ for scale S. The result is therefore the exact optimum of the dense problem. Wherever SciPy's dense solver fits, both found the same optimum: on 600 small instances and on all ten distributions at 10^4 points. Started from ShuffleSnap's layout, it certified all 202 benchmark instances up to 2×10^5 points, and uniform instances up to 10^6, in 1 to 13 pricing rounds. Started from bisection instead, it failed on four of ten instances at 10^5 points.

Correctness checks. We checked three properties independently of the kernel. Every output of every method gave each point its own usable cell (all 1,217 completed runs of the main benchmark); the exact cost recorded after every phase was monotone in all 540 ShuffleSnap runs and ended at the independently recomputed cost; and the window solver matched brute-force enumeration and SciPy on random, tie-heavy and badly scaled matrices.

5 Experimental setup

All runs used one workstation with an Intel i9-12900KS (8 performance and 8 efficiency cores, 24 hardware threads) and 62 GB of memory. Single-thread runs were pinned to one performance core; multi-thread runs used all 24 threads. Every run was a fresh process after a warm-up call on a small instance. Its time includes all of the method's own work, such as cost matrices, trees, starts and internal normalization, but not generating the instance or checking the result. Memory is the rise of the peak resident set during the call. Design decisions used instance seed 0, and all results use held-out seeds 1–3.

The ten synthetic distributions are uniform, Gaussian, a 10-component anisotropic mixture like typical embeddings, two clusters with 90/10 mass, a thin ring, a noisy spiral, a checkerboard density, heavy tails, 16 heavily duplicated sites and a nearly degenerate line. The real data are UMAP embeddings [8] of all 70,000 MNIST and Fashion-MNIST [15] images. The targets are near-square exact-fit grids, rectangles with surplus cells, a 4:1 rectangle, a disk, a ring and a blocky, rectilinear S-shaped mask.

Each comparator received the same coordinates and cells, and every output was checked for validity independently. *RasterFairy*'s 2019 PyPI release orders points by integer-truncated coordinates, which discards sub-cell order unless inputs are scaled up; its current upstream version normalizes internally and solves leaf blocks of up to 200 points exactly, and is our representative setting. *BSP-OT* ran through its official Python binding with 16 and 64 merged plans. The binding exposes no seed, so repeated calls differ: the costs of the same instance with 1 and 24 threads differed by −8% to +13%, with no trend. We measure neighbourhood preservation separately, as the fraction of each point's 10 nearest input neighbours among its 10 nearest grid neighbours (kNN recall), and the fraction of a point's occupied neighbour cells that hold one of its 8 nearest input neighbours (lattice precision).

Table 1: Excess over the certified optimum (%) at $N = 100{,}000$, mean of three held-out instances, one thread. The last column is the certified solver's time, started from ShuffleSnap (instance seed 1); the bottom row is the median time of each method.

distribution	SS fast	BSP-OT 16	SS bal.	BSP-OT 64	SS qual.	RasterFairy	exact (s)
uniform	2.58	44.8	0.84	32.1	0.44	44.8	16.5
Gaussian	<0.001	0.055	<0.001	0.040	<0.001	0.006	55.7
Gaussian mixture	0.013	5.61	0.002	3.74	<0.001	16.7	94.6
90/10 clusters	0.050	0.032	0.012	0.019	<0.001	1.16	214
ring	0.005	1.16	0.001	0.95	<0.001	41.0	114
spiral	0.022	9.96	0.004	8.06	0.001	27.3	120
checkerboard	0.13	49.8	0.057	36.1	0.020	425.2	64.9
heavy tails	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	261
duplicates	0.15	12.3	0.038	5.44	0.013	49.0	1364
near-line	0.010	0.008	0.002	0.002	<0.001	19.4	253
median time (s)	0.77	0.71	2.33	2.85	6.67	3.00	117

6 Results

6.1 Quality and time at 10^5 points

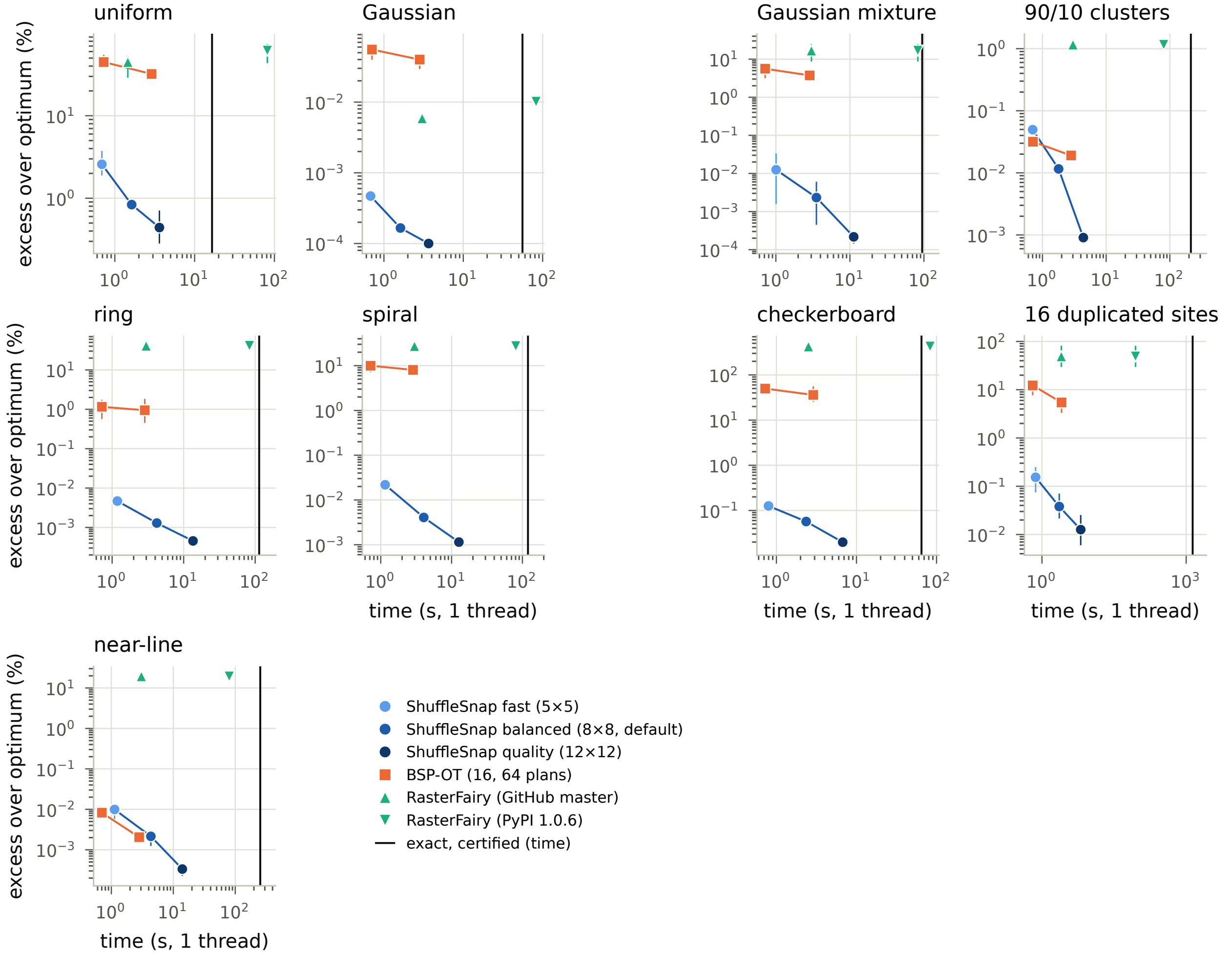

Figure 3: Excess over the certified optimum against single-thread time at $N = 100{,}000$ (mean over three held-out instances; bars span their range). The three ShuffleSnap presets form the blue curve, from fast (5×5) to quality (12×12). The black line is the time of the certified exact solver. Heavy tails are omitted: every method is below 0.001% there.

On nine of the ten distributions the default preset is within 0.06% of the optimum, and on six within 0.005% (Table 1, Figure 3). It needs a median of 2.3 s on one core. The exception is uniform points (0.84%). The presets trade time for quality on every distribution: fast (median 0.77 s) is within 0.16% on the same nine, and quality (6.7 s) lowers the uniform excess to 0.44%. At equal time ShuffleSnap beats BSP-OT: fast is closer to the optimum than BSP-OT with 16 plans (0.71 s) on eight of ten distributions, and the default than 64 plans (2.9 s) on nine, e.g. 2.6% and 0.84% against 45% and 32% on uniform points. BSP-OT is ahead only on the near-line and the 90/10 clusters. RasterFairy takes about 3 s and is 17–425% above the optimum on seven of the ten distributions; it does well on the Gaussian and the 90/10 clusters. With 24 threads, ShuffleSnap ran 7–14 times faster with bit-identical layouts (0.19 s for the default, 0.09 s for fast), while BSP-OT gained 4–7 times, because the cycle merge that combines its plans is largely sequential.

6.2 Matched budgets and scaling

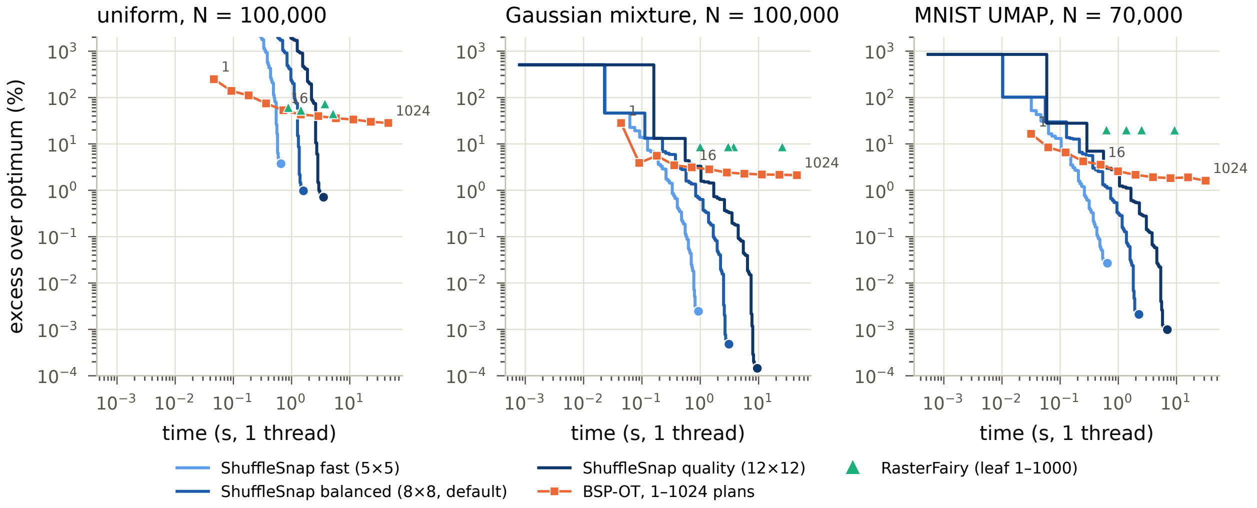

Figure 4: Excess over the certified optimum against elapsed time, one thread. ShuffleSnap's step curves record the exact cost after every phase, so each shows what a time budget ending at that moment would return. BSP-OT runs with 1 to 1024 plans; RasterFairy with exact leaf blocks of 1 to 1000 points.

BSP-OT improves slowly with more plans and levels off far from the optimum: from 1 to 1024 plans, its excess drops from 249% to 28% on uniform points and from 16% to 1.6% on MNIST (Figure 4). The fast ShuffleSnap preset passes BSP-OT's best result after 0.2–0.5 s and the default after 0.5–1.3 s. With less than 0.12–0.36 s on one thread, BSP-OT is the better choice, because ShuffleSnap has only finished its coarse levels; with 24 threads, the fast preset is ahead at every budget.

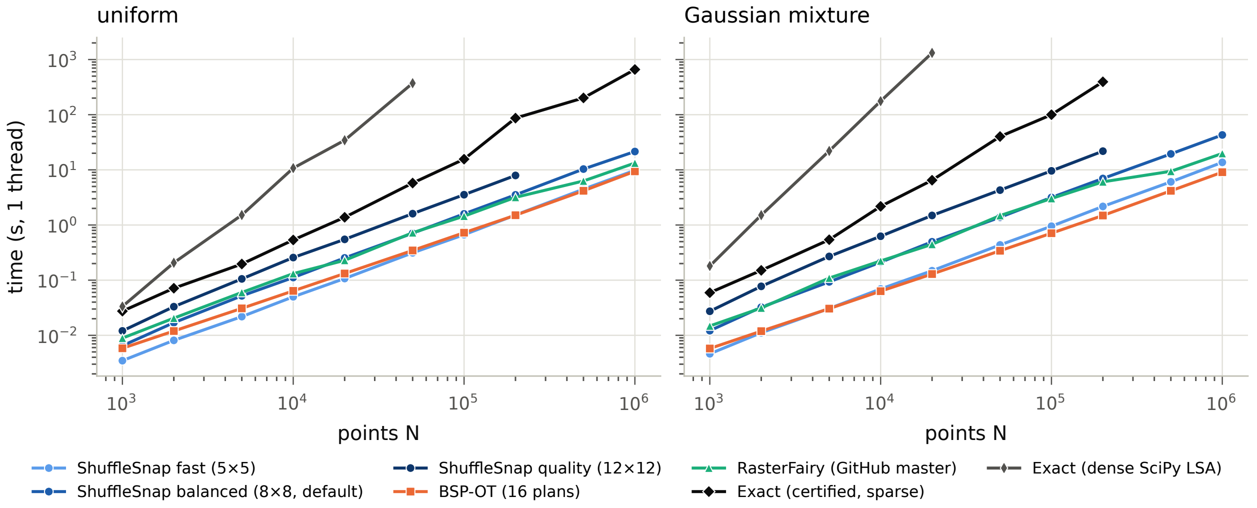

Figure 5: Time against N on one thread. Dense SciPy ran up to 50k uniform and 20k mixture points; the certified solver up to 10^6 uniform and 2×10^5 mixture points.

ShuffleSnap, BSP-OT and RasterFairy all scale nearly linearly (Figure 5): 10^6 uniform points take the fast preset 9.8 s on one core, as long as BSP-OT with 16 plans, and the default 21 s (1.5 s on 24 threads) with 55 bytes per point, a quarter to a twelfth of the comparators' memory. The certified solver took 11 min and 4.2 GiB there; dense SciPy needed 6 min and 23 GiB at 50,000. The default's uniform excess grows with N: 0.1% at 10^3, 0.84% at 10^5, 1.3% at 10^6.

6.3 Real embeddings: displacement versus neighbourhoods

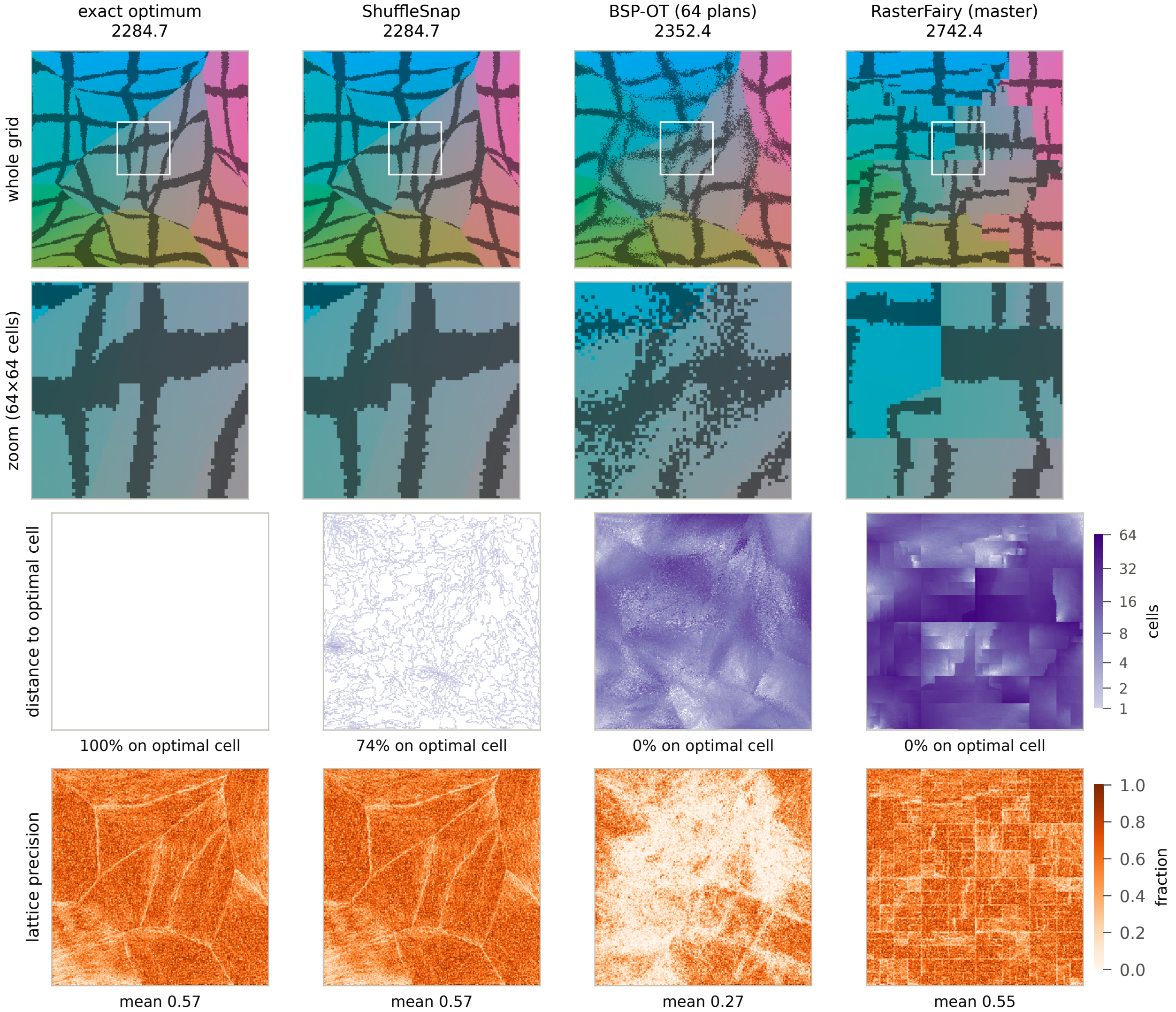

Figure 6: All 70,000 MNIST digits, UMAP layout snapped to a 265 × 265 grid. Colors and graph-paper lines follow each point's input position, so bent lines show how a method deforms the layout, and broken ones where it tears it; numbers are the mean squared displacement. Row 2 enlarges the white box, row 3 shows each point's distance to its cell in the optimum (log scale; white is the optimal cell), row 4 its lattice precision. ShuffleSnap leaves 74% of the points on their optimal cell; BSP-OT's median point is 8 cells away.

On both embeddings the default preset is within 0.002% of the certified optimum, the fast preset within 0.031% (BSP-OT with 16 plans: 3.7–9.2%), and their layouts match the optimal one (Figure 6). The two measures disagree on BSP-OT: with 64 plans its displacement is only 2.4% (MNIST) and 6.7% (Fashion-MNIST) above the optimum, yet kNN recall falls to 0.31–0.33 against 0.61–0.65, and lattice precision to about half. Its merged partitions are close to the transport map on average but shuffle points locally. RasterFairy shows the opposite: its displacement is 20–25% above the optimum, yet its neighbourhood measures are close to the optimum's, because local order survives inside each slice. Even the optimum tears the layout along cluster boundaries (pale lines in row 4). Minimizing displacement and preserving neighbourhoods are different goals.

6.4 Masks, surplus cells and design choices

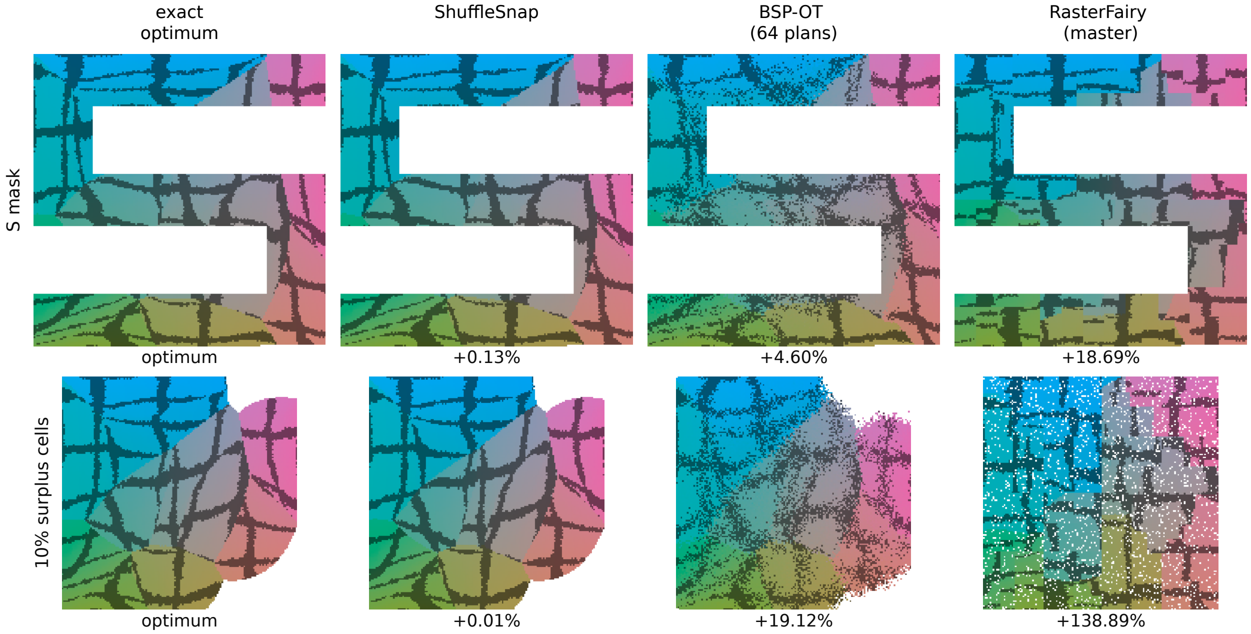

Figure 7: 20,000 points of the MNIST embedding on a rectilinear S-shaped mask and on a rectangle with 10% surplus cells; numbers are the excess over the certified optimum. With surplus cells, the optimum leaves one region empty, far from the data. BSP-OT's partial matching leaves ragged holes; RasterFairy removes surplus cells at random before slicing.

On disks and rings, the default stays within 0.07% of the optimum (fast 0.14%), while BSP-OT is 2.5–9.3% and RasterFairy 11–54% above it. On the S-shaped mask, coarse schedules can route mass across the bays. With uniform points the default reaches 1.4%, against 9.3% for BSP-OT with 64 plans, and quality 0.88%; this is the one mask input where the fast preset loses to BSP-OT with 16 plans (17% against 12%). Surplus cells, where the solver must choose the empty cells, are harder: the default stays within 0.91% with 10% surplus but reaches 3.1% on uniform points with less than a row of surplus. The comparators suffer most here: with 10% surplus, BSP-OT's partial matching is 18–109% above the optimum and RasterFairy 72–350%. For a prime number of points, an exact-fit grid with a partial last row is therefore simpler and better than a full rectangle for every method.

Varying one design choice at a time on held-out instances confirmed the development findings. The window side is the main quality knob: on uniform points the excess falls from 3.8% for 4×4 windows to 0.09% for 24×24, while the time grows 28-fold. Coarse levels are essential: capping the first stride made results up to 90 times worse and up to 45 times slower, because stride-1 windows alone stall far from the optimum. So are shifted tilings: with a single tiling no point can cross a window border (104% on uniform points), while two tilings give 2.0%, four 0.84% and nine 0.70% at twice the time. Skipping unchanged windows saves 22–24% of the time without changing any result; polishing at stride 2 after convergence changed almost nothing, and merging independent runs helped less per second than a larger window.

6.5 Anatomy of the remaining error

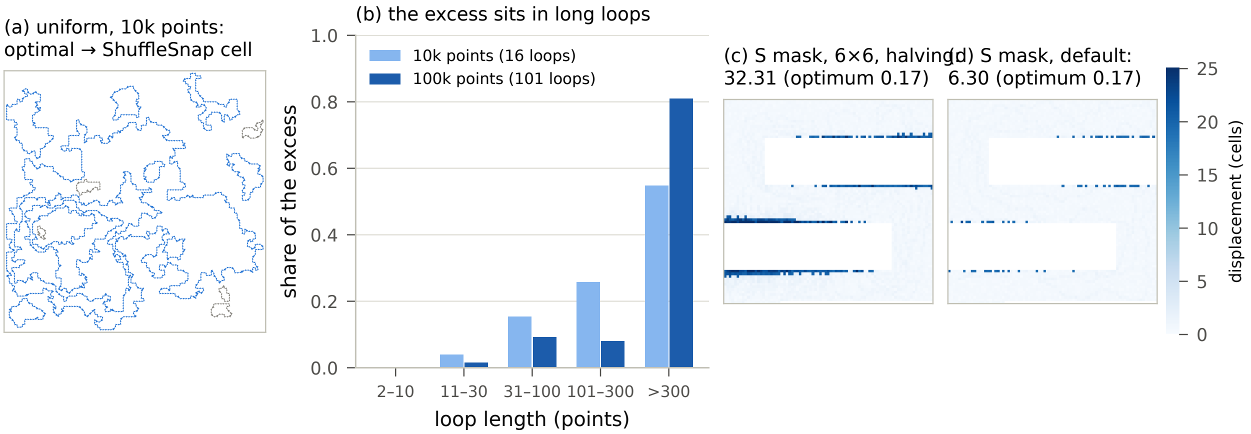

Figure 8: (a, b) Where ShuffleSnap and the optimum disagree on uniform points, the difference is a few long closed loops of one-cell moves (loops longer than 30 points in blue); at 10^5 points loops of more than 300 points carry most of the excess. (c, d) Points drawn inside the S mask (optimum $\approx$ identity): 6×6 windows with halving strides route points across the bays, where they line up; the default leaves fewer. Numbers are mean squared displacements.

Why does a method that solves every window exactly miss the optimum? The permutation that turns a ShuffleSnap layout into the optimal one splits into cycles, and almost all of the excess sits in a few very long ones, in which every point is shifted by about one cell (Figure 8a,b). Inside a window, part of such a loop is a run of consistently shifted points, which is already optimal for that occupant set, and a strided window cannot move a point by one cell. On points placed exactly on the cells, with an axis-aligned ring of cells rotated by one step, the full schedule never moves a ring larger than the window, so the relative excess over the optimum of zero is unbounded. The same mechanism appears at a larger scale on masks: coarse windows route mass across concave bays, which no stride-1 window can undo (Figure 8c,d), and a mask can even cut two usable cells off from each other when no scheduled window contains both.

7 Discussion

Coarse-to-fine strided windows get very close to the optimum on the smooth, clustered layouts that motivate the problem, in a fraction of a second with all cores at 10^5 points, deterministically and with little memory. They fail where a local search must: on uniform data, lattices and surplus cells, whose errors need moves no window can see. A billion points converged in 1 h 43 min using 53 GiB, passing a six-hour run of an earlier implementation after 44 minutes; a single-plan BSP-OT ended 8.2% above. Built as an evaluation tool, the exact solver proved optimality at 10^5 points in a median of about two minutes when started from ShuffleSnap. When a proven optimum is worth that wait, it is the better choice; otherwise the default preset is the fast, near-optimal answer, and BSP-OT is best only under budgets below 0.12–0.36 s on one core. If neighbourhoods matter, they should be optimized or measured directly, for example with LAS/FLAS [1].

Limitations. Everything ran on one workstation, CPU only (BSP-OT's GPU version was not tested), for squared Euclidean displacement in 2D, on two real UMAP embeddings, with representative settings of each comparator. The package is on PyPI as `shufflesnap`; raw results and figure scripts are in a separate repository.

References

[1] Kai Uwe Barthel, Nico Hezel, Klaus Jung, and Konstantin Schall. Improved evaluation and generation of grid layouts using distance preservation quality and linear assignment sorting. *Computer Graphics Forum*, 42(1):261–276, 2023. doi: 10.1111/cgf.14718. URL `https://doi.org/10.1111/cgf.14718`.

[2] Yann Brenier. Polar factorization and monotone rearrangement of vector-valued functions. *Communications on Pure and Applied Mathematics*, 44(4):375–417, 1991. doi: 10.1002/cpa.3160440402. URL `https://doi.org/10.1002/cpa.3160440402`.

[3] Baptiste Genest, Nicolas Bonneel, Vincent Nivoliers, and David Coeurjolly. BSP-OT: Sparse transport plans between discrete measures in loglinear time. *ACM Transactions on Graphics*, 44(6):1–15, December 2025. doi: 10.1145/3763281. URL `https://doi.org/10.1145/3763281`. Proc. SIGGRAPH Asia 2025.

[4] R. Jonker and A. Volgenant. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, 38(4):325–340, 1987. doi: 10.1007/BF02278710. URL `https://doi.org/10.1007/BF02278710`.

[5] Mario Klingemann. RasterFairy. `https://github.com/Quasimondo/RasterFairy`, 2015. URL `https://github.com/Quasimondo/RasterFairy`. GitHub repository created 2016-01-22; latest PyPI release of `rasterfairy` is 1.0.6 (2019-04-29). Accessed 2026-09-23.

[6] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955. doi: 10.1002/nav.3800020109. URL `https://doi.org/10.1002/nav.3800020109`.

[7] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791. URL `https://doi.org/10.1109/5.726791`.

[8] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction, 2018. URL `https://arxiv.org/abs/1802.03426`.

[9] Laurent Perron and Vincent Furnon. OR-Tools. `https://developers.google.com/optimization/`, 2026. URL `https://developers.google.com/optimization/`. Version v9.15, released 2026-01-12. Accessed 2026-09-23.

[10] Gabriel Peyré and Marco Cuturi. Computational optimal transport with applications to data sciences. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019. doi: 10.1561/2200000073. URL `https://doi.org/10.1561/2200000073`.

[11] Bernhard Schmitzer. A sparse multiscale algorithm for dense optimal transport. *Journal of Mathematical Imaging and Vision*, 56(2):238–259, 2016. doi: 10.1007/s10851-016-0653-9. URL `https://doi.org/10.1007/s10851-016-0653-9`.

[12] Donald L. Shell. A high-speed sorting procedure. *Communications of the ACM*, 2(7):30–32, 1959. doi: 10.1145/368370.368387. URL `https://doi.org/10.1145/368370.368387`.

[13] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008. URL `https://jmlr.org/papers/v9/vandermaaten08a.html`.

[14] A. Volgenant. Linear and semi-assignment problems: A core oriented approach. *Computers & Operations Research*, 23(10):917–932, 1996. doi: 10.1016/0305-0548(96)00010-X. URL `https://doi.org/10.1016/0305-0548(96)00010-X`.

[15] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017. URL `https://arxiv.org/abs/1708.07747`.